

PROPATIA Calculation Audit

This document lists the calculations currently performed in the application, page by page. It is intended for verification only. It explains formulas using non-private sample numbers instead of exporting live tenant, invoice, or payment data.

Generated PDF target: 'RENTALS/docs/propatia_calculation_audit.pdf'

How To Read This Audit

Each item includes:

- Displayed field: The number or status shown in the app.
- Formula: The exact calculation logic.
- Source: The code location that performs the calculation.
- Explanation: A short reason for the calculation.
- Example: A small hand-checkable example.

Dashboard Page

Source page: 'dashboard/templates/dashboard/dashboard.html'

Displayed field: Total Properties

- Formula: 'count(properties owned by current user)'
- Source: 'dashboard/views.py:index'
- Explanation: Counts the landlord/admin user's properties.
- Example: If the user owns Green Court and Blue Court, total properties = '2'.

Displayed field: Total Tenants

- Formula: 'count(tenants where tenant.unit.property.user == current user)'
- Source: 'dashboard/views.py:index'
- Explanation: Counts tenants attached to units in the user's properties.
- Example: If Green Court has 6 tenants and Blue Court has 4 tenants, total tenants = '6 + 4 = 10'.

Displayed field: Total Units

- Formula: 'sum(Property.total_units for current user's properties)'
- Source: 'dashboard/views.py:index'
- Explanation: Uses the declared total unit count stored on each property.
- Example: Green Court total units = 20, Blue Court total units = 12, total units = '20 + 12 = 32'.

Displayed field: Recent Property Occupancy

- Formula: 'occupied_units = count(property.units where status == "occupied")'
- Source: 'dashboard/views.py:index'
- Explanation: Shows occupied units for each recent property.
- Example: If a property has 12 units and 9 are marked occupied, display = '9/12 units occupied'.

Displayed field: Monthly Revenue

- Formula: 'sum(Payment.amount where payment.user == current user and payment.date >= today - 30 days)'
- Source: 'dashboard/views.py:index'
- Explanation: Adds all payments received in the last 30 days.
- Example: Payments in last 30 days are KES 8,000, KES 12,000, and KES 5,000. Monthly revenue = '8,000 + 12,000 + 5,000 = 25,000'.

Displayed field: Previous-Month Revenue

- Formula: 'sum(Payment.amount where today - 60 days <= payment.date < today - 30 days)'
- Source: 'dashboard/views.py:index'
- Explanation: Adds payments from the previous 30-day window for comparison.
- Example: Payments in the previous window are KES 9,000 and KES 6,000. Previous-month revenue = '9,000 + 6,000 = 15,000'.

Displayed field: Revenue Increase Percentage

- Formula: '((monthly_revenue - previous_month_revenue) / previous_month_revenue) * 100', rounded to 1 decimal place. If previous-month revenue is zero, the percentage is '0'.
- Source: 'dashboard/views.py:index'
- Explanation: Measures the percentage change between the current 30-day revenue and the previous 30-day revenue.

- Example: Monthly revenue = KES 25,000, previous-month revenue = KES 15,000. Increase = $((25,000 - 15,000) / 15,000) * 100 = 66.7\%$.

Reports Page

Source page: 'reports/templates/reports/report_view.html'

Displayed field: Total Units

- Formula: 'count(units after selected property/date filters are applied)'
- Source: 'reports/views.py:report_generator'
- Explanation: Counts units belonging to the current user, optionally narrowed by property.
- Example: If the selected property has 18 units, total units = '18'.

Displayed field: Occupied Units

- Formula: 'count(filtered units where status == "occupied")'
- Source: 'reports/views.py:report_generator'
- Explanation: Counts filtered units that are currently occupied.
- Example: Total filtered units = 18, occupied units = '12'.

Displayed field: Vacant Units

- Formula: 'total_units - occupied'
- Source: 'reports/views.py:report_generator'
- Explanation: Vacant units are whatever remains after occupied units are removed from the filtered unit count.
- Example: Total units = 18, occupied = 12, vacant = '18 - 12 = 6'.

Displayed field: Rent Expected

- Formula: 'sum(Invoice.amount where Invoice.type == "Rent" after filters)'
- Source: 'reports/views.py:report_generator'
- Explanation: Adds rent invoices in the selected report scope.
- Example: Rent invoices are KES 12,000, KES 12,000, and KES 10,000. Rent expected = '12,000 + 12,000 + 10,000 = 34,000'.

Displayed field: Rent Collected

- Formula: 'sum(Payment.amount after filters)'
- Source: 'reports/views.py:report_generator'
- Explanation: Adds payment amounts in the selected report scope.
- Example: Payments are KES 8,000, KES 10,000, and KES 6,000. Rent collected = '8,000 + 10,000 + 6,000 = 24,000'.

Displayed field: Water Total

- Formula: 'sum(WaterBill.amount after filters)'
- Source: 'reports/views.py:report_generator'
- Explanation: Adds water bill amounts in the selected report scope.
- Example: Water bills are KES 500, KES 650, and KES 450. Water total = '500 + 650 + 450 = 1,600'.

Displayed field: Collection Rate

- Formula: '(rent_collected / rent_expected) * 100', rounded to 1 decimal place. If rent expected is zero, collection rate is '0'.
- Source: 'reports/views.py:report_generator'
- Explanation: Shows how much of expected rent was collected.
- Example: Rent collected = KES 80,000, rent expected = KES 100,000. Collection rate = '(80,000 / 100,000) * 100 = 80%'.

Tenant Ledger Page

Source page: 'tenants/templates/tenants/tenant_ledger.html'

Displayed field: Invoice Debit

- Formula: 'debit = Invoice.amount'
- Source: 'tenants/views.py:tenant_ledger'
- Explanation: Every invoice increases the tenant balance.
- Example: Invoice amount = KES 12,000, debit = '12,000'.

Displayed field: Payment Credit

- Formula: 'credit = Payment.amount'
- Source: 'tenants/views.py:tenant_ledger'
- Explanation: Every payment reduces the tenant balance.
- Example: Payment amount = KES 5,000, credit = '5,000'.

Displayed field: Running Balance

- Formula: 'running_balance = previous_balance + debit - credit'
- Source: 'tenants/views.py:tenant_ledger'
- Explanation: Ledger entries are sorted by date. Each invoice adds to the balance and each payment subtracts from it.
- Example: Previous balance = KES 7,000, next invoice debit = KES 12,000, next payment credit = KES 5,000. Running balance = '7,000 + 12,000 - 5,000 = 14,000'.

Displayed field: Total Invoiced

- Formula: 'sum(Invoice.amount for tenant)'
- Source: 'tenants/views.py:tenant_ledger'
- Explanation: Adds all invoices for that tenant.
- Example: Invoices are KES 12,000, KES 12,000, and KES 3,000. Total invoiced = '27,000'.

Displayed field: Total Paid

- Formula: 'sum(Payment.amount for tenant)'
- Source: 'tenants/views.py:tenant_ledger'
- Explanation: Adds all payments recorded for that tenant.
- Example: Payments are KES 10,000 and KES 8,000. Total paid = '18,000'.

Displayed field: Current Balance

- Formula: 'total_invoiced - total_paid'
- Source: 'tenants/views.py:tenant_ledger'
- Explanation: Shows the tenant's remaining balance across all invoice and payment records.
- Example: Total invoiced = KES 27,000, total paid = KES 18,000. Current balance = '27,000 - 18,000 = 9,000'.

Displayed field: Overdue Invoice Count

- Formula: 'count(invoices where due_date < today and status != "Paid")'
- Source: 'tenants/views.py:tenant_ledger'
- Explanation: Counts invoices that are past due and not fully paid.
- Example: Four invoices exist; two are past due and unpaid. Overdue invoice count = '2'.

Invoices Page

Source page: 'invoices/templates/invoices/invoices_view.html'

Displayed field: Generated Invoice Amount

- Formula: 'invoice.amount = unit.rent_amount'
- Source: 'invoices/views.py:invoice_list'
- Explanation: A generated rent invoice uses the selected unit's rent amount.
- Example: Unit A1 rent amount = KES 12,000. Generated invoice amount = '12,000'.

Displayed field: Amount Paid

- Formula: 'sum(InvoicePayment.amount_applied for invoice)'
- Source: 'invoices/models.py:Invoice.get_amount_paid'
- Explanation: Adds all payment allocations attached to that invoice.
- Example: Allocations are KES 5,000 and KES 2,000. Amount paid = '5,000 + 2,000 = 7,000'.

Displayed field: Balance Due / Remaining Balance

- Formula: 'invoice.amount - invoice.get_amount_paid()'
- Source: 'invoices/models.py:Invoice.get_remaining_balance'
- Explanation: The remaining balance is the invoice total minus allocations already applied.
- Example: Invoice amount = KES 12,000, amount paid = KES 5,000. Balance due = '12,000 - 5,000 = 7,000'.

Displayed field: Invoice Status

- Formula: 'Paid if remaining <= 0; Partially Paid if remaining < invoice.amount; otherwise Unpaid'
- Source: 'invoices/models.py:Invoice.update_status'
- Explanation: Status follows the remaining balance.
- Example: Invoice amount = KES 12,000 and remaining = KES 0, status = 'Paid'. If remaining = KES 4,000, status = 'Partially Paid'.

Displayed field: Payment Amount Applied

- Formula: 'amount_applied must be > 0 and <= min(payment.balance, invoice.remaining_balance)'
- Source: 'invoices/views.py:attach_payment_to_invoice'
- Explanation: A payment cannot apply more credit than the payment has or more than the invoice still needs.
- Example: Payment balance = KES 8,000 and invoice remaining balance = KES 5,000. Maximum allowed amount applied = 'min(8,000, 5,000) = 5,000'.

Displayed field: Payment Balance After Attachment

- Formula: 'payment.balance = payment.balance - amount_applied'
- Source: 'invoices/views.py:attach_payment_to_invoice';
'invoices/services.py:_apply_payment_to_invoice'
- Explanation: Allocating part of a payment reduces its available credit.
- Example: Payment balance = KES 8,000 and KES 5,000 is applied. New payment balance = '8,000 - 5,000 = 3,000'.

Displayed field: Edited Payment Allocation Difference

- Formula: 'difference = new_amount_applied - old_amount_applied'; then 'payment.balance = payment.balance - difference'

- Source: 'invoices/views.py:update_invoice_payment'
- Explanation: Increasing an allocation consumes more payment credit; decreasing it restores credit.
- Example: Old allocation = KES 4,000, new allocation = KES 6,000, difference = '2,000'. Payment balance is reduced by KES 2,000.

Displayed field: Removed Payment Allocation

- Formula: 'payment.balance = payment.balance + removed_amount_applied'
- Source: 'invoices/views.py:remove_invoice_payment'
- Explanation: Removing an allocation returns that amount to the payment as available credit.
- Example: Removed allocation = KES 3,000 and payment balance was KES 1,000. New balance = '1,000 + 3,000 = 4,000'.

Displayed field: Automatic Rent Payment Allocation

- Formula: 'amount_applied = min(payment.balance, invoice.remaining_balance)' applied to oldest open rent invoices first.
- Source: 'invoices/services.py:allocate_payment_to_rent_invoices'
- Explanation: A rent payment is consumed against the oldest unpaid rent invoice until either invoices are cleared or payment credit runs out.
- Example: Payment balance = KES 20,000. Oldest invoice needs KES 12,000 and next invoice needs KES 12,000. First allocation = KES 12,000, remaining payment balance = KES 8,000, second allocation = KES 8,000.

Displayed field: Existing Credit Applied To New Rent Invoice

- Formula: 'amount_applied = min(payment.balance, new_invoice.remaining_balance)' for tenant payments with credit.
- Source: 'invoices/services.py:allocate_credit_to_rent_invoice'
- Explanation: When a rent invoice is created, existing unallocated tenant credit can automatically reduce it.
- Example: New invoice = KES 12,000 and tenant credit = KES 5,000. Applied credit = '5,000', invoice remaining = '7,000'.

Payments Page

Source page: 'payments/templates/payments/payments_view.html'

Displayed field: Payment Starting Balance

- Formula: 'payment.balance = payment.amount' when the payment is first created and no balance was supplied.
- Source: 'payments/models.py:Payment.save'
- Explanation: A new payment starts as fully unallocated credit.
- Example: Payment amount = KES 8,000. Starting balance = '8,000'.

Displayed field: Allocated Amount

- Formula: 'sum(InvoicePayment.amount_applied for payment)'
- Source: 'payments/views.py:payment_list'
- Explanation: Adds the parts of this payment that have been attached to rent invoices.
- Example: A payment is split into KES 5,000 and KES 3,000 allocations. Allocated amount = '8,000'.

Displayed field: Remaining Credit

- Formula: 'remaining_credit = payment.balance'
- Source: 'payments/views.py:payment_list'
- Explanation: Shows the unallocated part of the payment.
- Example: Payment amount = KES 10,000 and allocated amount = KES 7,000. Remaining credit = '3,000'.

Displayed field: Payment Status

- Formula: 'claimed if balance <= 0; otherwise unclaimed'
- Source: 'invoices/services.py:set_payment_status_from_balance'
- Explanation: Fully allocated payments are claimed; payments with leftover credit remain unclaimed.
- Example: Payment balance = KES 0, status = 'claimed'. Payment balance = KES 2,000, status = 'unclaimed'.

Displayed field: Allocation Summary

- Formula: 'join(each allocation invoice month and amount_applied)'
- Source: 'payments/views.py:payment_list'
- Explanation: Shows which invoice months a payment covers.
- Example: Allocations are May 2026 KES 5,000 and Jun 2026 KES 3,000. Summary = 'May 2026: KES 5,000, Jun 2026: KES 3,000'.

Displayed field: Uploaded Payment Amount

- Formula: Remove commas/currency symbols and parse the remaining number as Decimal. Parentheses are treated as negative.
- Source: 'payments/views.py:upload_payments'
- Explanation: Allows payment imports with values like 'KES 9,200.00'.
- Example: Uploaded value 'KES 9,200.00' becomes '9200.00'.

Displayed field: Payment Tenant Resolution

- Formula: The selected unit must belong to the selected property and must have an active lease. The active lease tenant is assigned to the payment.
- Source: 'payments/views.py:resolve_active_lease_tenant'
- Explanation: Prevents assigning payments to the wrong unit or tenant.

- Example: Property = Green Court, unit = A1, active lease tenant = Jane Doe. Payment tenant = 'Jane Doe'.

Arrears Report Page

Source page: 'arrears/templates/arrears/arrears_report.html'

Displayed field: Arrears Amount Due

- Formula: 'invoice.get_remaining_balance()' for overdue unpaid or partially paid invoices.
- Source: 'arrears/views.py:sync_user_arrears'
- Explanation: Arrears are based on the unpaid part of an overdue invoice, not always the original invoice amount.
- Example: Invoice amount = KES 12,000 and KES 5,000 was paid. Arrears amount due = '12,000 - 5,000 = 7,000'.

Displayed field: Days Overdue

- Formula: '(today - invoice.due_date).days'
- Source: 'arrears/views.py:sync_user_arrears'
- Explanation: Counts calendar days since the invoice due date.
- Example: Today is July 15 and due date was July 5. Days overdue = '10'.

Displayed field: Total Arrears Amount

- Formula: 'sum(Arrears.amount_due after filters)'
- Source: 'arrears/views.py:arrears_report'
- Explanation: Adds all filtered arrears records.
- Example: Arrears amounts are KES 7,000, KES 3,000, and KES 10,000. Total arrears = '20,000'.

Displayed field: Total Records

- Formula: 'count(filtered arrears records)'
- Source: 'arrears/views.py:arrears_report'
- Explanation: Counts how many arrears rows match the selected filters.
- Example: Five arrears rows match the selected property. Total records = '5'.

Displayed field: Monthly Arrears

- Formula: 'sum(amount_due grouped by tenant, unit, and invoice due month)'
- Source: 'arrears/views.py:arrears_report'
- Explanation: Groups arrears by tenant/unit/month to show month-level unpaid rent.
- Example: Tenant Jane has two June arrears records of KES 4,000 and KES 3,000. June monthly arrears = '4,000 + 3,000 = 7,000'.

Displayed field: Accumulated Arrears

- Formula: 'previous accumulated arrears for tenant + current monthly arrears'
- Source: 'arrears/views.py:arrears_report'
- Explanation: Builds a running arrears total per tenant across months.
- Example: Tenant's May accumulated arrears = KES 5,000 and June monthly arrears = KES 7,000. June accumulated arrears = '5,000 + 7,000 = 12,000'.

Displayed field: Resolved Arrears

- Formula: If an arrears record is no longer linked to an active overdue invoice, mark it resolved.
- Source: 'arrears/views.py:sync_user_arrears'; 'arrears/models.py:Arrears.mark_resolved'
- Explanation: Arrears clear when the invoice is fully paid or no longer overdue/open.

- Example: Invoice remaining balance becomes KES 0. The arrears record is marked 'resolved'.

Water Bills Page

Source page: 'water_bills/templates/water_bills/water_bills_view.html'

Displayed field: Consumption

- Formula: 'current_reading - previous_reading'
- Source: 'water_bills/models.py:WaterBill.save'
- Explanation: Water consumption is the difference between the current and previous meter readings.
- Example: Current reading = 120, previous reading = 100. Consumption = '120 - 100 = 20'.

Displayed field: Water Bill Amount

- Formula: 'consumption * rate'
- Source: 'water_bills/models.py:WaterBill.save'
- Explanation: The bill amount is units consumed multiplied by the rate per unit.
- Example: Consumption = 20 and rate = KES 50. Amount = '20 * 50 = 1,000'.

Displayed field: Water Bill Amount Paid

- Formula: 'sum(WaterBillPaymentAllocation.amount_applied for water bill)'
- Source: 'water_bills/models.py:WaterBill.get_amount_paid'
- Explanation: Adds all water payment allocations attached to the bill.
- Example: Allocations are KES 400 and KES 300. Amount paid = '700'.

Displayed field: Water Bill Remaining Balance

- Formula: 'water_bill.amount - water_bill.get_amount_paid()'
- Source: 'water_bills/models.py:WaterBill.get_remaining_balance'
- Explanation: Shows the unpaid part of the water bill.
- Example: Water bill amount = KES 1,000 and amount paid = KES 700. Remaining balance = '300'.

Displayed field: Water Bill Status

- Formula: 'Paid if remaining_balance <= 0; otherwise Unpaid'
- Source: 'water_bills/models.py:WaterBill.update_status'
- Explanation: A water bill is paid only when all of it has been allocated.
- Example: Remaining balance = KES 0, status = 'Paid'. Remaining balance = KES 300, status = 'Unpaid'.

Displayed field: Uploaded Water Bill Consumption Validation

- Formula: 'uploaded consumption must equal current_reading - previous_reading'
- Source: 'water_bills/views.py:bulk_generate_water_bills'
- Explanation: Prevents importing a row where the consumption column disagrees with the meter readings.
- Example: Previous = 100, current = 120, expected consumption = '20'. Uploaded consumption must be '20'.

Displayed field: Uploaded Water Bill Amount Validation

- Formula: 'uploaded amount must equal consumption * rate'
- Source: 'water_bills/views.py:bulk_generate_water_bills'
- Explanation: Prevents importing a row with a wrong bill amount.
- Example: Consumption = 20 and rate = KES 50, expected amount = '1,000'. Uploaded amount must be '1,000'.

Water Bill Payments Page

Source page: 'water_bills/templates/water_bills/water_bill_payments_view.html'

Displayed field: Water Payment Starting Balance

- Formula: 'payment.balance = payment.amount' when first created and no balance was supplied.
- Source: 'water_bills/models.py:WaterBillPayment.save'
- Explanation: A new water payment starts as unallocated credit.
- Example: Payment amount = KES 1,500. Starting balance = '1,500'.

Displayed field: Water Payment Allocation

- Formula: 'amount_applied = min(payment.balance, water_bill.remaining_balance)' for open water bills ordered by due date.
- Source: 'water_bills/views.py:allocate_payment_to_water_bills'
- Explanation: Water payments are allocated to the oldest unpaid water bill first.
- Example: Payment balance = KES 1,500 and oldest bill remaining = KES 1,000. Amount applied = '1,000', payment balance becomes '500'.

Displayed field: Water Payment Allocated Amount

- Formula: 'sum(WaterBillPaymentAllocation.amount_applied for payment)'
- Source: 'water_bills/views.py:water_bill_payments'
- Explanation: Adds all water bill allocations made from this payment.
- Example: Allocations are KES 1,000 and KES 300. Allocated amount = '1,300'.

Displayed field: Water Payment Remaining Credit

- Formula: 'remaining_credit = payment.balance'
- Source: 'water_bills/views.py:water_bill_payments'
- Explanation: Shows the water payment amount that has not yet been allocated.
- Example: Payment amount = KES 1,500 and allocated amount = KES 1,300. Remaining credit = '200'.

Displayed field: Water Payment Status

- Formula: 'claimed if balance <= 0; otherwise unclaimed'
- Source: 'water_bills/views.py:set_water_payment_status_from_balance'
- Explanation: Fully used water payments are claimed; partially unused payments remain unclaimed.
- Example: Balance = KES 0, status = 'claimed'. Balance = KES 200, status = 'unclaimed'.

Displayed field: Deleted Water Bill Allocation Restore

- Formula: 'payment.balance = payment.balance + allocation.amount_applied'
- Source: 'water_bills/views.py:delete_water_bills'
- Explanation: If a water bill is deleted, its allocations are returned to the water payment credit balance.
- Example: Payment balance = KES 200 and deleted allocation = KES 800. New payment balance = '1,000'.

Leases, Units, And Tenants Pages

Source pages: 'leases/templates/leases/leases_view.html',
'units/templates/units/units_view.html', 'tenants/templates/tenants/tenants_view.html'

Displayed field: Lease Monthly Rent

- Formula: 'lease.monthly_rent = lease.unit.rent_amount' when the lease is first created.
- Source: 'leases/models.py:Lease.save'; 'units/views.py:assign_unit'
- Explanation: The lease stores a rent snapshot from the unit at lease creation time.
- Example: Unit rent amount = KES 12,000. New lease monthly rent = '12,000'.

Displayed field: Deposit Held

- Formula: 'deposit_held = user-entered deposit amount on lease assignment'
- Source: 'units/views.py:assign_unit'; 'leases/models.py:Lease.deposit_held'
- Explanation: Deposit held is not calculated from rent in the current code; it is stored from the submitted value.
- Example: User enters deposit held = KES 12,000. Lease deposit held = '12,000'.

Displayed field: Tenant Deposit Amount

- Formula: 'deposit_amount = user-entered tenant deposit amount'
- Source: 'tenants/models.py:Tenant.deposit_amount'; 'tenants/forms.py:TenantForm'
- Explanation: Tenant deposit amount is stored directly; there is no automatic rent-based calculation in the current code.
- Example: User enters KES 10,000. Tenant deposit amount = '10,000'.

Displayed field: Unit Occupied/Vacant Status

- Formula: 'occupied when an active lease exists for the unit; vacant when no other active lease remains'
- Source: 'leases/models.py:update_unit_status_on_active_lease';
'units/models.py:Unit.is_occupied'
- Explanation: Unit occupancy follows active tenant/lease assignment.
- Example: Unit A1 receives an active lease, status becomes 'occupied'. If the lease becomes inactive and no other active lease exists, status becomes 'vacant'.

Displayed field: Unit Rent Amount From Upload

- Formula: Remove commas from uploaded rent amount and parse as Decimal. Some unquoted comma splits are repaired before parsing.
- Source: 'units/views.py:upload_units'
- Explanation: Allows imports such as '15,000.00' to be stored as a numeric rent amount.
- Example: Uploaded rent amount '15,000.00' becomes '15000.00'.

Displayed field: Occupancy Counts Used Elsewhere

- Formula: 'count(Unit where status == "occupied")'; vacant counts are commonly 'total_units - occupied'
- Source: 'dashboard/views.py:index'; 'reports/views.py:report_generator'
- Explanation: The dashboard and reports rely on unit status to compute occupancy.
- Example: Total units = 20 and occupied units = 13. Vacant units = '20 - 13 = 7'.

Important Verification Notes

- Money values are stored as Decimal fields in the database for payments, invoices, units, leases, arrears, and water bills.
- Display formatting such as 'floatformat:2' changes how values look, not how they are calculated.
- Most page filters are applied before totals are calculated, so changing property, unit, status, or date filters changes the totals.
- Payment allocation is intentionally limited by two balances: available payment credit and invoice/bill remaining balance.
- The PDF does not include live private data; examples are artificial and only demonstrate arithmetic.